





CENTRO DE INFORMÁTICA

DESAFIO DE DEV PTC 26.2

CITi

Aspirante:
KAIO RUAN DA SILVA BARROS - krsb@cin.ufpe.br

Recife, 2026.

SUMÁRIO

0. RESUMO PARA OS AVALIADORES.....3
1. O QUE FOI ENTREGUE.....3
2. COMO O TRABALHO FOI DIVIDIDO.....4
3. ETAPA POR ETAPA: PROBLEMAS E SOLUÇÕES..... 5
5. USO DE INTELIGÊNCIA ARTIFICIAL..... 10

0. RESUMO PARA OS AVALIADORES

Construí a API REST da Folhagem & Cia, a loja de plantas que controlava o estoque numa planilha e "vendeu uma samambaia que não existia mais no estoque". A entrega tem **8 rotas** (as 7 obrigatórias e 1 bônus), os **4 bônus**, **105 testes automatizados** passando e **14 commits** pequenos, já na main pelo merge. O documento da API está no README.md, como pede o DESAFIO.md.

O ponto central foi a pergunta do enunciado sobre duas pessoas comprando o último vaso ao mesmo tempo. Primeiro provei o problema com uma versão ingênua da venda, que chegou a vender **20 vasos com 1 em estoque** (estoque final −19). Depois resolvi com um desconto atômico no banco, que vendeu **1 vaso e recusou as outras 19 compras**. Eu não conhecia nenhuma das tecnologias do projeto, então o trabalho foi dividido em 12 etapas, de baixo para cima, cada uma testada antes do commit.

1. O QUE FOI ENTREGUE

A API substitui a planilha da loja e segue a mesma organização em camadas do CRUD de usuários do template (rotas, Controller e Repository), com uma camada a mais de validação.

Tabela 1 – Rotas da API de plantas

MÉTODO	ROTA	O QUE FAZ	SUCESSO
POST	/plantas	Cadastra uma planta	201
GET	/plantas	Lista as plantas, com os filtros ?luminosidade= e ?petFriendly=, sozinhos ou juntos	200
GET	/plantas/:id	Busca uma planta	200
PATCH	/plantas/:id	Edita só os campos enviados	200
DELETE	/plantas/:id	Remove uma planta	204
POST	/plantas/:id/venda	Recebe { "quantidade": n } e diminui o estoque	200
GET	/plantas/estoque/valor-total	Soma preço × estoque de todas as plantas e o total de vasos	200
GET	/plantas/esgotadas	Lista as plantas com estoque zero (bônus)	200

Fonte: minha elaboração, a partir do routes.ts e do README.md do repositório.

Todas as regras de negócio do enunciado foram implementadas e testadas. A Tabela 2 mostra cada regra e como ela é garantida no código.

Tabela 2 – Regras de negócio e como cada uma é garantida

REGRA DO ENUNCIADO	COMO FOI GARANTIDA
Todos os campos são obrigatórios no cadastro; na edição, nenhum é, mas os enviados seguem as mesmas regras	Uma única função, validarPlanta(corpo, parcial), atende o cadastro e a edição. Todas as colunas são NOT NULL no banco
preco maior que zero; regaACadaDias de pelo menos 1	Number.isInteger mais o intervalo mínimo e máximo, com mensagens que mostram o intervalo válido
estoque não pode ser negativo, mas pode ser zero	Intervalo de 0 até o limite do Int. A validação compara com undefined e nunca usa !campo, que recusaria o zero
luminosidade só aceita os três valores do enum	A validação usa Object.values(Luminosidade), e o próprio banco recusa outro valor (tipo ENUM)
Dados inválidos no cadastro, na edição, nos filtros e na venda → 400 com mensagem clara	Formato único { "message", "erros": [...] }, com todos os problemas listados de uma vez
id que não existe → 404	A planta é buscada antes de editar, remover ou vender. Um id mal formado (abc, 0, 1.5) recebe 400 antes de qualquer consulta

REGRA DO ENUNCIADO	COMO FOI GARANTIDA
A venda nunca deixa o estoque negativo e não altera nada se faltar estoque	A conferência e o desconto acontecem numa única operação atômica no banco, provada com compras simultâneas (Etapa 6)

Fonte: minha elaboração, com as regras do DESAFIO.md e o código do PlantaValidator e do PlantaController.

Fiz os quatro bônus, só depois de todo o obrigatório estar pronto e testado (busca por nome sem diferenciar maiúsculas (?nome=costela), ordenação por preço ou nome (?ordem=), a rota GET /plantas/esgotadas e um seed com 12 plantas reais (yarn seed) que não duplica dados).

Tratei o JSON malformado, que antes devolvia uma página HTML com os caminhos do servidor, deixei os 105 testes versionados no repositório (yarn test) e fiz um teste do zero, clonando o repositório numa pasta nova (essa parte não estava nas obrigações, mas vi e achei interessante resolver).

Nenhum requisito obrigatório nem bônus ficou de fora. Só existiram limitações conhecidas, também documentadas no README, como a busca que diferencia acentos (adao não encontra “adão”; resolver exigiria a extensão unaccent do PostgreSQL) e, na resposta de uma venda, o estoque mostrado pode já incluir outra venda feita no mesmo instante, embora o estoque gravado no banco esteja sempre correto.

2. COMO O TRABALHO FOI DIVIDIDO

Segui a sequência sugerida no enunciado, construindo de baixo para cima: o dado, o acesso ao banco, a validação e as rotas. Cada etapa terminava com um commit, para que nenhuma camada fosse construída em cima de outra que ainda não estava testada. A Tabela 3 é o mapa das etapas e do principal problema de cada uma; o detalhe está na seção 3.

Tabela 3 – Etapas do desenvolvimento

ETAPA	O QUE FOI FEITO	PRINCIPAL PROBLEMA	COMMIT
0	Ambiente e estudo do CRUD de usuários	Não conhecia a stack; tinha clonado o repositório dos instrutores	Nenhum
1	Model Planta, enum e migration	Prisma Client do container desatualizado	1
2	Repository (acesso ao banco)	Testar sem rotas; arquivos com quebra de linha do Windows	2
3	Validação	!campo recusaria estoque 0 e petFriendly false	3
4	Controller do CRUD e rotas	Commit feito antes de uma correção	4
5	Filtros	Filtro ausente virando false	5
6	Venda atômica	Duas compras simultâneas do último vaso	6
7	Valor total do estoque	O aggregate não multiplica duas colunas	7
8	Bônus e seed	Curingas do ILIKE; npm num projeto Yarn	8 9
9	JSON malformado, testes e teste do zero	Teste instável; comando destrutivo na pasta errada	10 11 12
10	Código limpo e documento da API	Garantir que o README é fiel à API	13 14
11	Pull Request e merge na main	Levar o código à main sem perder o histórico	15

Fonte: minha elaboração, com o histórico de commits do repositório.

A requisição passa pelo routes.ts, que liga cada URL a uma função do PlantaController. O Controller chama o PlantaValidator para conferir id, corpo, filtros e venda, aplica as regras

de negócio e escolhe o status HTTP. O PlantaRepository é a única camada que fala com o Prisma e o PostgreSQL. O Validator é a única camada que não existia no modelo; separei a validação para deixar o Controller curto e poder testá-la sozinha, sem banco nem servidor.

Ciclo de cada etapa. Todas as etapas seguiram o mesmo ciclo:

1. Eu escrevia o código.
2. Testes no meu computador.
3. Commit e push.

Esse ciclo pegou vários detalhes antes que virassem problema, como arquivos salvos com quebra de linha do Windows e um commit que levou junto, por engano, uma mudança de dependência.

3. ETAPA POR ETAPA: PROBLEMAS E SOLUÇÕES

Para cada etapa, apresento o que foi feito e, em seguida, uma tabela com os problemas que apareceram, como cada um foi resolvido e o que aprendi com ele. As dificuldades estão no ponto em que realmente aconteceram.

Etapa 0 – Ambiente e entendimento do projeto. Instalei e conferi Git, Node.js 20, Yarn, Docker Desktop, rodei o passo a passo do README e estudei o CRUD de usuários arquivo por arquivo, com 13 testes manuais. Não houve commit de código.

Tabela 4 – Etapa 0: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Eu não conhecia Node.js, TypeScript, Express, Prisma, PostgreSQL nem Docker	Comparei o que eu previa lendo o código com o que acontecia de verdade, em 13 testes manuais no modelo de usuário	A API roda em container e os scripts no Windows (o cross-env troca o host); o ts-node-dev não checa tipos, por isso o yarn typecheck
Achei que “todos os POST davam 500”	Li o log com docker compose logs e achei o código P2002: eu tinha enviado o mesmo documento duas vezes	Ler um erro do Prisma em três partes (tipo, mensagem e pilha) e ver que um 500 genérico não ajuda o cliente
O modelo de usuário respondia 500 para enum inválido e id abc, e uma página HTML para JSON malformado	Anotei para fazer diferente nas plantas: validar antes do banco (400) e tratar o JSON malformado (feito na Etapa 9)	Todo erro causado pelo cliente deve virar 400 com uma mensagem útil
Eu tinha clonado o repositório dos instrutores, e não um meu	Criei o meu pelo “Use this template” e reposicionei as branches com git checkout -B	Só trocar o origin não bastava: o template cria um histórico novo, e o Pull Request seria impossível

Fonte: minha elaboração, a partir dos 13 testes manuais e dos logs da etapa.

Etapa 1 – Model e migration. Criei o model Planta e o enum Luminosidade com os nomes de campo exatamente iguais aos do enunciado e gerei a migration create_planta_model. O preço ficou como inteiro em centavos, porque 0.1 + 0.2 não dá 0.3 em ponto flutuante, e nenhum campo ganhou valor padrão, para que um campo esquecido pelo cliente não passasse despercebido.

Tabela 5 – Etapa 1: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Uma migration aplicada não se edita: se o model estivesse errado, seria preciso outra migration	Rodei prisma format e prisma validate antes de aplicar, li o SQL gerado e conferi com migrate status, \d "Planta", \dT+ e o Prisma Studio	A migration é o histórico do banco; erro se corrige com uma nova migration
O schema tinha comentários de estudo explicando cada linha	Os comentários viraram anotações minhas; no código ficaram só os /// com regra de negócio	Comentário em código explica o porquê, e não o óbvio
Docker fechado, psql abrindo um paginador e Studio mostrando só a tabela User	Abri o Docker Desktop, usei -P pager=off e reabri o Studio, que era uma instância antiga	Conferir o ambiente antes de suspeitar do código
O Prisma Client dentro do container não foi regenerado (o --watch não vê mudanças feitas no Windows)	Confirmei com grep -c Planta no client do container (0); depois de docker compose restart server, passou a 1. A regra foi para o README	Existem dois Prisma Clients, o do editor e o da API, e eles podem ficar dessincronizados

Fonte: minha elaboração, a partir das conferências feitas no banco e nos logs.

Etapa 2 – Repository. Escrevi as cinco funções do CRUD (createPlanta, findAllPlantas, findPlantaById, updatePlanta e deletePlanta) no mesmo estilo do UserRepository. O Repository não valida nem trata "não encontrado".

Tabela 6 – Etapa 2: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Ainda não havia rotas para testar as funções	Fiz um teste de fumaça com um script temporário dentro do container: 6/6. findPlantaById(999999) devolveu null e o update do preço manteve o estoque	Os tipos do Prisma já carregam as regras: CreateInput exige tudo, UpdateInput deixa tudo opcional
Sem ORDER BY, a lista poderia vir em qualquer ordem	Usei orderBy: { id: "asc" } na listagem	O PostgreSQL não garante a ordem do resultado sem ORDER BY

Fonte: minha elaboração, a partir do teste de fumaça da etapa.

Etapa 3 – Validação. É a etapa que decide o status de cada caso de erro. Organizei a validação em duas perguntas por campo: o campo foi enviado? E o conteúdo está certo? A diferença entre o POST e o PATCH está só na primeira, então uma única função, validarPlanta(corpo, parcial), atende os dois. Testei a validação sozinha com 35 casos.

Tabela 7 – Etapa 3: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
if (!estoque) recusaria uma planta esgotada (0) e petFriendly false	"Veio?" é respondido com === undefined; o conteúdo é conferido por tipo e intervalo (Number.isInteger, typeof, trim, lista do enum)	Separar "foi enviado?" de "está certo?"
Um PATCH { "preço": 9990 } (com ç) seria ignorado e daria 200 sem mudar nada	Campos desconhecidos são recusados com 400, e um PATCH com {} também	Erro silencioso é pior que erro explícito
Números acima de 2.147.483.647 dariam 500 no banco, e o Number() aceita "1e3" e "0x10" como id	Limite do Int do PostgreSQL nos números e no id, e id validado com a expressão /^\d+$/	Validar antes do banco para que nenhum erro do cliente vire 500
Preço acima do limite recebia "deve ser maior que zero"	As mensagens passaram a mostrar o intervalo válido	Os testes revelam também mensagens ruins, não só código errado

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
O includes sugerido pelo enunciado é recusado pelo TypeScript quando o valor é unknown	Troquei por .some(), que faz a mesma comparação	unknown obriga a checar o tipo antes de usar o valor

Fonte: minha elaboração, a partir da tabela de 35 casos de validação.

Etapa 4 – Controller do CRUD e rotas. Todo handler segue a mesma ordem: valida o id (400), valida o corpo (400), confere se a planta existe (404) e só então chama o Repository e responde. Validar antes de buscar faz a entrada inválida dar sempre 400. Testei com 29 testes HTTP automáticos.

Tabela 8 – Etapa 4: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Um PATCH só do preço quebraria ao chamar nome.trim() com nome undefined	Usei encadeamento opcional: nome?.trim()	Na edição, todo campo pode não ter vindo
Riscos apontados no enunciado: “Cannot set headers after they are sent” e req.body indo direto ao banco	return antes de cada res.status e desestruturação do req.body	Nunca mandar ao banco o que o cliente enviou sem filtrar

Fonte: minha elaboração, a partir da bateria de 29 testes HTTP.

Etapa 5 – Filtros. A listagem passou a aceitar ?luminosidade= e ?petFriendly=, sozinhos ou juntos. A bateria cadastra quatro plantas, uma de cada combinação, e confere a lista exata de cada filtro. Fiz 15 casos de teste.

Tabela 9 – Etapa 5: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Na URL tudo chega como texto: ?petFriendly=true é a string "true"	Converto o texto para booleano, mas só quando o filtro foi enviado	Query string nunca traz tipos
Com req.query.petFriendly === "true", um filtro ausente viraria false e GET /plantas listaria só as não pet-friendly	No where do Prisma, uma condição undefined é ignorada; uma única consulta atende as quatro combinações	undefined (“não informado”) é diferente de false (“informado, e é falso”)
Quem digitasse ?petfriendly=true (f minúsculo) receberia com 200 todas as plantas, inclusive as tóxicas	Filtros desconhecidos dão 400; filtro válido sem resultado dá 200 [], nunca 404	Pensar no usuário final, que tem um gato em casa

Fonte: minha elaboração, a partir da bateria de 15 testes de filtros.

Etapa 6 – Venda e o problema do último vaso. A venda valida o id e a quantidade, busca a planta (404), confere o estoque (400 sem alterar nada), desconta e devolve a planta atualizada. O enunciado deixou uma pergunta: e se duas pessoas comprarem o último vaso ao mesmo tempo? Os 17 testes da etapa passaram.

Tabela 10 – Etapa 6: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
“Ler o estoque, conferir e descontar” são duas operações: duas compras podem ler “1 vaso” antes de qualquer desconto, e o estoque fica negativo	Conferência e desconto numa única instrução: updateMany com where { id, estoque: { gte: quantidade } } e decrement. Se nenhuma linha muda, a resposta é 400	No Node, cada await é uma pausa em que outras requisições entram; é uma condição de corrida

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Provar que o problema existia, em vez de supor	Escrevi de propósito uma versão ingênua, diferente da final em uma linha, e disparei compras simultâneas com Promise.all (Tabela 11)	Ver o bug acontecer e desaparecer convence mais que a teoria
Numa primeira tentativa, com uma rodada só, a versão ingênua passou por sorte	Cada cenário passou a ser repetido 5 vezes	Bugs de concorrência são intermitentes; um teste que passa uma vez não prova nada
A operação atômica sozinha não diz quantos vasos faltam	Mantive a conferência anterior para a mensagem precisa ("há apenas 3 em estoque"); a condição atômica é a rede de segurança	Considerei SELECT ... FOR UPDATE e CHECK (estoque >= 0), mas fiquei com a solução mais simples que resolve

Fonte: minha elaboração, a partir do teste de venda e concorrência.

Tabela 11 – Compras simultâneas antes e depois da correção (5 rodadas por cenário)

CENÁRIO	VERSÃO INGÊNUA	VERSÃO ATÔMICA (ENTREGUE)
20 compras simultâneas, 1 vaso em estoque	até **20 vendidas**; estoque **−19**	1 vendida, 19 recusadas; estoque **0**
50 compras simultâneas, 10 vasos em estoque	até **42 vendidas**; estoque **−32**	10 vendidas, 40 recusadas; estoque **0**

Fonte: minha elaboração, com o script de teste que dispara as compras ao mesmo tempo (Promise.all).

Etapa 7 – Valor total do estoque. A rota GET /plantas/estoque/valor-total soma preço × estoque de todas as plantas. Busquei só as duas colunas com select e fiz a conta no Controller com reduce. Os testes conferem que o total acompanha venda, mudança de preço e remoção. Fiz 7 casos de teste.

Tabela 12 – Etapa 7: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
O aggregate do Prisma soma uma coluna, mas não multiplica duas	select de preco e estoque mais reduce no Controller	Saber o limite da ferramenta antes de forçá-la
Loja vazia	O reduce com valor inicial devolve 200 com zeros	Lista vazia não é erro
Conferir se a conta estava certa	Refiz à mão: 8.990 × 12 + 4.590 × 0 + 7.490 × 8 + 3.990 × 20 = **247.600 centavos** (R$ 2.476,00), 40 vasos	Conta de dinheiro se confere à mão, não se supõe
A ordem das rotas importava aqui?	Testei movendo a rota para depois de /plantas/:id e ela continuou funcionando; mesmo assim, registrei antes por convenção	O :id captura um único segmento da URL, e esta rota tem dois

Fonte: minha elaboração, a partir dos 7 testes de valor total.

Etapa 8 – Bônus. Implementei a busca por nome, a ordenação, a rota de esgotadas e o seed com 12 plantas reais (as três luminosidades, plantas pet-friendly e tóxicas segundo a ASPCA, e duas esgotadas, entre elas a Samambaia). Foram 21 testes.

Tabela 13 – Etapa 8: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
?nome=% trazia a loja inteira: o Prisma não escapa os curingas do ILIKE	Achei numa sondagem no banco antes de escrever o código e criei escaparCuringas, que trata %, _ e a barra como texto	Sondar a ferramenta antes de confiar nela
Duas plantas com o mesmo preço podiam trocar de posição entre consultas	id como critério de desempate na ordenação	Ordenação estável precisa de desempate

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Registrada depois de /:id, a rota esgotadas respondia 400 “O id deve ser um número inteiro positivo”	Provei o erro e registrei /plantas/esgotadas antes de /plantas/:id	Aqui a ordem das rotas importa, porque “esgotadas” é um único segmento
O seed automático do Prisma rodaria a cada migrate reset e quebraria o “zerar o banco” dos testes	Criei o yarn seed como script próprio, que avisa e não faz nada se já houver plantas. Total conferido: 606.280 centavos, 142 vasos	Seed não pode destruir nem duplicar dados

Fonte: minha elaboração, a partir dos 21 testes de bônus e do git status da etapa.

Etapa 9 – Acabamento técnico e teste do zero. Tratei o JSON malformado, transformei os scripts de teste das etapas anteriores numa bateria versionada (testes/bateria-completa.mjs, executada com yarn test) e fiz o teste do zero num clone novo. Fiz 105 casos de teste.

Tabela 14 – Etapa 9: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Com JSON malformado, o Express devolvia uma página HTML com a pilha de erro e os caminhos do servidor	Tratador de erros em server.ts: JSON inválido → 400 em JSON; corpo acima de 100 KB → 413; outro erro → 500 genérico	O Express reconhece um tratador de erros pelos 4 parâmetros, por isso o next fica mesmo sem uso
Os testes estavam em scripts temporários, fora do repositório	105 testes em 7 suítes, com o banco zerado antes de cada suíte e código de saída 1 se algo falhar	O avaliador confere em cerca de 15 segundos o que o documento afirma
Teste instável: a suíte do seed falhou, passou quatro vezes e falhou de novo com fetch failed num clone	Investiguei em três passos (formato da falha, o que funcionou e o que falhou, hipótese). A bateria repete só GETs, uma vez, e mostra quantas repetições houve	Teste instável se investiga, não se roda até passar; a hipótese da conexão reaproveitada não foi confirmada nem descartada
Rodei docker compose down -v na pasta do projeto original, e não na do clone, e apaguei o banco de desenvolvimento	Desliguei os containers do clone, que têm nomes fixos, e recriei o banco com yarn migration e yarn test	Conferir a pasta antes de qualquer comando destrutivo

Fonte: minha elaboração, a partir das execuções do yarn test no projeto e no clone.

Etapa 10 – Polimento e documento da API. Revisei o projeto contra o checklist de entrega e escrevi o documento da API no README.md.

Tabela 15 – Etapa 10: problemas e soluções

PROBLEMA	COMO FOI RESOLVIDO	O QUE APRENDI
Garantir código limpo: sem console.log de teste, código comentado ou import sem uso	grep no projeto, tsc --noUnusedLocals, yarn typecheck e diff dos arquivos do template; a pasta .vscode/ foi para o .gitignore	Os parâmetros req e next sem uso são exigidos pela assinatura do Express, não são código morto
O documento precisava ser “fiel ao que a API realmente faz”	Exemplos capturados da API rodando, 20 de 20 mensagens de erro conferidas contra o código, contas refeitas e links do sumário testados	Documentar só o que foi conferido

Fonte: minha elaboração, a partir da revisão de código e do README.md.

Etapa 11 – Entrega. Abri o Pull Request da branch desafio/kaio-barros para a main, com uma descrição que resume a entrega e o checklist, e fiz o merge.

5. USO DE INTELIGÊNCIA ARTIFICIAL

Usei o Claude (Anthropic) durante o desafio, de forma pontual e não constante. Usei, inicialmente, para aprender mais sobre as linguagens e como funciona trabalhar com rotas de API. Após isso, utilizei o Claude como um revisor e tutor: eu escrevia o código da etapa (ele me auxiliou na divisão das etapas também), ele avaliava se precisava mudar algo e eu pedia a elaboração de testes para testar o que tinha feito. O README.md foi organizado por ele e escrito em cada etapa por mim.

Os 13 testes do modelo de usuário, os 35 casos de validação, as baterias HTTP e os 105 testes finais foram feitos pela IA para eu conseguir saber se existia algo errado (em alguns deles eu encontrava erros e consertava então foi bastante produtivo).

A IA acelera muito o aprendizado de uma stack nova, mas o valor está em entender e verificar, e não em copiar. As partes que mais me ensinaram foram as que deram errado.

APÊNDICE – TESTES

Tabela A1 – Os 105 testes automatizados por suíte

SUÍTE	TESTES	O QUE COBRE
CRUD	29	Todos os casos de CRUD da tabela do enunciado; PATCH inválido não altera nada; id no corpo recusado; id acima do Int; lista ordenada
Filtros	15	Cada filtro sozinho e juntos; petFriendly=false; sem resultado → []; valores inválidos, vazios, repetidos e desconhecidos
Venda e concorrência	17	Vender o que tem e mais do que tem; planta esgotada; quantidades inválidas; 20 compras para 1 vaso e 50 para 10 vasos (5 rodadas)
Valor total	7	Loja vazia; conta à mão; acompanhamento de venda, edição e remoção
Bônus	21	Busca (maiúsculas, trecho, espaços, % e _); ordenação (empate, com filtros, inválida); esgotadas
JSON malformado	11	Vírgula sobrando, chave sem aspas, texto solto; PATCH e venda; CRUD de usuários; corpo > 100 KB (413)
Seed	5	12 plantas; não duplica; listagem; valor total (606.280 / 142); esgotadas
Total	**105**	**105/105 no projeto e em clones limpos da branch e da main**

Fonte: minha elaboração, feita com a descrição da IA sobre os testes que ela fez.